

EduGenie

Smart Learning Assistant using Google Gemini

Project Report

EduGenie is a lightweight AI-powered educational assistant designed to simplify learning through Google Gemini generative AI. It provides question answering, concept explanations, quiz generation, text summarization, and personalized learning recommendations through a simple web interface.

Technology Stack

- Python 3.10+
- FastAPI
- HTML + CSS
- Jinja2
- Google Gemini API / Google GenAI SDK
- Uvicorn
- Python-dotenv

1. Project Description

EduGenie is a smart AI-powered learning assistant that uses Google Gemini to help students learn in an interactive and accessible way. Instead of requiring students to search through multiple sources for every doubt, the system provides a single interface for common learning tasks.

The application is designed for students of different academic levels. A learner can ask a question, request a simplified explanation, generate a quiz, summarize educational material, or request a structured learning path.

The project combines a lightweight FastAPI backend with a simple HTML and CSS frontend. Google Gemini performs the generative AI tasks while the backend controls the flow of information between the user interface and the AI modules.

Key Objectives

- Provide quick and student-friendly answers.
- Simplify difficult academic concepts.
- Support active learning through quizzes.
- Reduce long passages into revision-friendly summaries.
- Provide structured learning recommendations.
- Demonstrate practical integration of generative AI with a web application.

2. Scenarios and Use Cases

Scenario 1 - Question Answering

A student wants to learn about oceans and rivers and asks: "Which is the largest ocean?" EduGenie sends the question to Gemini and displays a concise educational answer.

Scenario 2 - Quiz Generation

A student is studying the Pythagoras Theorem and wants to check their understanding. The learner selects the Generate Quiz option. EduGenie creates three multiple-choice questions with four options and identifies the correct answer.

Scenario 3 - Learning Path

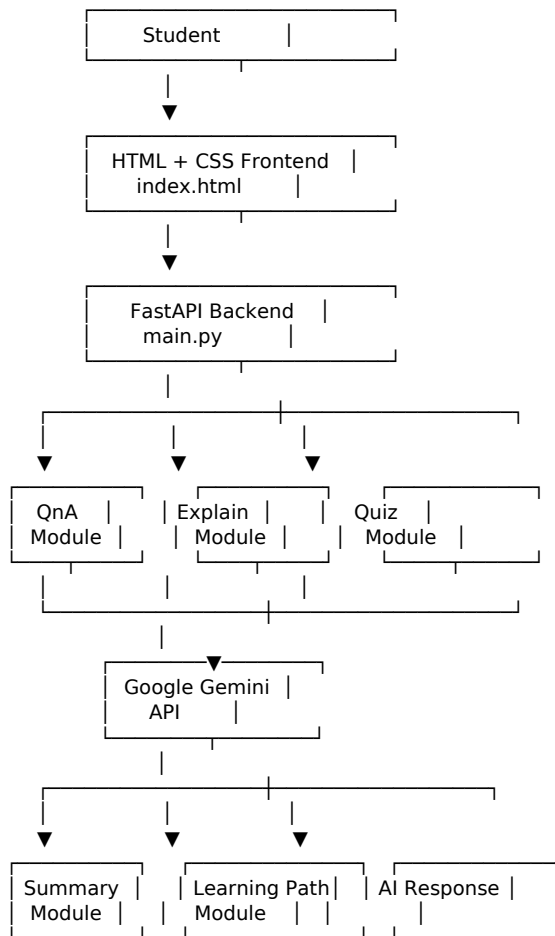
A learner who is beginning SQL can request a learning path. EduGenie generates beginner, intermediate, and advanced stages, along with suggested timelines, practice activities, learning resources, and a final assessment or project.

Target Users

- School and college students
- Beginners learning programming and technical subjects
- Learners preparing for revision and self-testing
- Students who need difficult concepts explained in simpler language

3. Architectural Diagram

The architecture separates the presentation layer, backend processing layer, individual AI modules, and Google Gemini service. This modular design makes the application easier to understand, test, and extend.



The student interacts only with the frontend. The FastAPI application receives the selected task and forwards the input to the corresponding module. Each module prepares an educational prompt and calls Google Gemini.

4. Pre-requisites

Python 3.10+

Python is the main programming language used to implement the backend and AI modules. Install a recent Python version and verify it using the terminal command below.

```
python --version
```

FastAPI Framework

FastAPI is used to create the backend web application and REST-style routes. It is lightweight and supports modern Python development.

```
pip install fastapi
```

Uvicorn

Uvicorn acts as the ASGI server used to run the FastAPI application.

```
pip install uvicorn
```

Jinja2

Jinja2 provides HTML templating so that the backend can send generated results to the frontend.

```
pip install jinja2
```

Python-dotenv

Python-dotenv loads environment variables such as the Gemini API key from a .env file.

```
pip install python-dotenv
```

5. Google Gemini API and GenAI SDK

EduGenie uses Google Gemini as the core generative AI model. The application sends carefully written prompts and user content to Gemini and receives generated educational responses.

Google AI Studio can be used to create or manage a Gemini API key. The key should be stored securely in an environment file rather than directly inside Python source code.

Example environment variable

```
GEMINI_API_KEY=your_api_key_here
```

Install the Google GenAI SDK

```
pip install google-genai
```

Main AI capabilities

- Question answering
- Concept explanation
- Quiz generation
- Text summarization
- Personalized learning paths

Security Note

The API key is a sensitive credential. It should not be committed to public repositories, placed directly in frontend HTML, or shared in screenshots. A production system should also use appropriate secret-management and access controls.

6. Folder Architecture

```
EduGenie/
├── main.py
├── explanation_module.py
├── qna.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
├── .env
├── templates/
│   └── index.html
└── static/
    └── style.css
```

File responsibilities

- main.py – FastAPI application, routes, form handling, and response rendering.
- qna.py – question-answering logic.
- explanation_module.py – simplified concept explanation.
- quiz_module.py – generation and formatting of multiple-choice questions.
- summary_module.py – summarization of educational text.
- learning_path.py – personalized learning recommendations.
- templates/index.html – frontend form and result display.
- static/style.css – visual design and responsive styling.
- requirements.txt – Python dependencies.
- .env – local environment variable containing the Gemini API key.

7. Explanation Module

The Explanation Module converts difficult topics into student-friendly explanations. The prompt requests a simple definition, step-by-step explanation, important points, and an example.

```
import os
from dotenv import load_dotenv
from google import genai

load_dotenv()

client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)

def explain_topic(topic):
    prompt = f"""
    You are EduGenie, a smart educational tutor.

    Explain the following topic in simple language:

    {topic}

    Provide:
    1. Simple definition
    2. Step-by-step explanation
    3. Important points
    4. Simple example

    Make the explanation easy for students to understand.
    """

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )

    return response.text
```

This module is especially useful when a learner understands the subject name but needs a simpler explanation. Prompt structure helps guide Gemini toward an educational response rather than an unrestricted general answer.

8. QnA Module

The QnA module handles general academic questions. It creates a concise prompt that identifies Gemini as an educational assistant and requests a clear, accurate, student-friendly answer.

```
def answer_question(question):  
    prompt = f"""  
    You are EduGenie, an AI educational assistant.  
  
    Answer the student's question clearly and accurately.  
  
    Question:  
    {question}  
  
    Give a concise and easy-to-understand answer.  
    """  
  
    response = client.models.generate_content(  
        model="gemini-2.5-flash",  
        contents=prompt  
    )  
  
    return response.text
```

Example workflow

Input: Which is the largest ocean? → FastAPI receives the form → qna.py prepares the prompt → Gemini generates the response → main.py renders the result in index.html.

Advantages

- Simple interface for academic doubts
- Short and readable responses
- Reusable module structure
- Easy integration with the main FastAPI application

9. Quiz Module

The Quiz Module turns educational content into three multiple-choice questions. Each question contains four options and one correct answer. JSON is requested so that the application can parse and format the result.

```
def clean_json_block(text):
    text = text.strip()

    if text.startswith("```json"):
        text = text[7:]
    if text.startswith("```"):
        text = text[3:]
    if text.endswith("```"):
        text = text[:-3]

    return text.strip()

def generate_quiz(content):
    prompt = f"""
Generate exactly 3 multiple-choice questions
from the following educational content.

Content:
{content}

Return ONLY valid JSON in this format:

[
  {{
    "question": "Question",
    "options": ["A", "B", "C", "D"],
    "answer": "Correct answer"
  }}
]
"""

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )

    cleaned_response = clean_json_block(response.text)
    quiz = json.loads(cleaned_response)
    return quiz
```

The `clean_json_block` function helps remove Markdown code fences when the model returns JSON inside a fenced block. Error handling should be retained in the final implementation so malformed model output does not crash the application.

10. Summary Module

The Summary Module is designed for revision. Students can paste a long educational passage and receive a shorter version that retains important information while removing unnecessary detail.

```
def summarize_text(text):  
    prompt = f"""  
    You are EduGenie, an educational summarization assistant.  
  
    Summarize the following content:  
  
    {text}  
  
    Requirements:  
    - Keep important information  
    - Remove unnecessary details  
    - Use simple language  
    - Use bullet points where appropriate  
    - Make it useful for quick revision  
    """  
  
    response = client.models.generate_content(  
        model="gemini-2.5-flash",  
        contents=prompt  
    )  
  
    return response.text
```

Benefits for students

- Faster revision
- Simplified reading
- Important points are easier to identify
- Useful for lengthy notes and educational passages
- Can support last-minute topic review

11. Learning Path Module

The Learning Path Module generates a structured plan for a selected subject. For example, when the learner enters SQL, the model can organize the topic into beginner, intermediate, and advanced stages.

```
def get_learning_recommendations(topic):

    prompt = f"""
    You are EduGenie, a personalized learning assistant.

    Create a structured learning path for:

    {topic}

    Include:
    1. Beginner level
    2. Intermediate level
    3. Advanced level
    4. Important concepts
    5. Suggested timeline
    6. Practice activities
    7. Learning resources
    8. Final project or assessment

    Make the plan practical and student-friendly.
    """

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )

    return response.text
```

The output can be adapted to a student's selected topic. It can include weekly targets, practice tasks, and a final project so that learning becomes more structured.

12. Backend API with FastAPI

FastAPI connects the frontend form to the five educational modules. The /process route receives two form fields: task and text. Based on the task, it invokes the appropriate function.

```
app = FastAPI(
    title="EduGenie - Smart Learning Assistant"
)

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(
        "index.html",
        {"request": request, "result": None}
    )

@app.post("/process", response_class=HTMLResponse)
async def process(
    request: Request,
    task: str = Form(...),
    text: str = Form(...)
):
    if task == "qa":
        result = answer_question(text)
    elif task == "explain":
        result = explain_topic(text)
    elif task == "quiz":
        result = generate_quiz(text)
    elif task == "summarize":
        result = summarize_text(text)
    elif task == "learn":
        result = get_learning_recommendations(text)
    else:
        result = "Invalid task selected."

    return templates.TemplateResponse(
        "index.html",
        {
            "request": request,
            "result": result,
            "selected_task": task,
            "user_text": text
        }
    )
```

Health endpoint

```
@app.get("/health")
async def health():
    return {"status": "EduGenie is running"}
```

13. Frontend Development

The frontend is intentionally simple. It contains a task dropdown, a text area, a submit button, and a result container. This keeps the interface approachable for students.

```
<form method="post" action="/process">

  <label>Select Task</label>

  <select name="task" required>
    <option value="qa">Ask a Question</option>
    <option value="explain">Explain a Topic</option>
    <option value="quiz">Generate Quiz</option>
    <option value="summarize">Summarize</option>
    <option value="learn">Recommend Learning Path</option>
  </select>

  <label>Enter your question or content</label>

  <textarea
    name="text"
    rows="8"
    placeholder="Enter your question, topic or content..."
    required></textarea>

  <button type="submit">Ask EduGenie</button>

</form>
```

The result section is conditionally displayed when the backend sends a generated result. Jinja2 is used to connect the Python backend with the HTML template.

14. CSS Styling and User Experience

The CSS creates a clean educational interface with a centered container, card-style sections, readable typography, input controls, and a prominent action button.

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, sans-serif;
  background: #f4f7fb;
  color: #222;
}

.container {
  width: 90%;
  max-width: 900px;
  margin: 40px auto;
}

.card, .result {
  background: white;
  padding: 25px;
  border-radius: 12px;
  margin-bottom: 25px;
  box-shadow: 0 4px 15px rgba(0,0,0,0.08);
}

select, textarea {
  width: 100%;
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 8px;
  font-size: 16px;
}

button {
  width: 100%;
  margin-top: 20px;
  padding: 13px;
  border: none;
  border-radius: 8px;
  font-size: 17px;
  cursor: pointer;
}
```

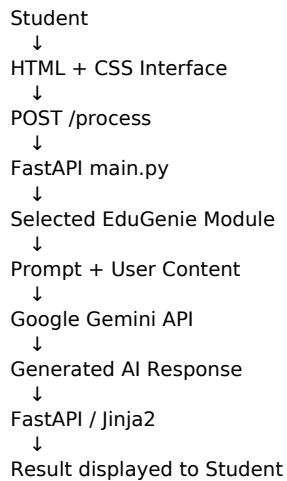
Responsive layout

The container uses a percentage width with a maximum width, allowing the page to adapt to different screen sizes. The input fields also use the full available width, which improves usability on smaller displays.

15. Complete Project Workflow

The complete workflow begins when the student opens the EduGenie webpage. The frontend presents the available tasks. After the learner selects a task and enters content, the browser sends a POST request to the FastAPI /process route.

FastAPI identifies the selected task and invokes the corresponding module. The module prepares a prompt and sends it to Google Gemini through the GenAI SDK. Gemini generates the response and returns it to the module. The result is then passed back to the Jinja2 template and displayed to the student.



This flow keeps the frontend simple while allowing the backend to manage AI-related processing.

16. Installation and Deployment

Install all dependencies with the requirements file.

```
pip install -r requirements.txt
```

Run the application locally using Uvicorn.

```
uvicorn main:app --reload
```

Open the local application in a browser at:

```
http://127.0.0.1:8000
```

requirements.txt

```
fastapi
uvicorn
jinja2
python-dotenv
google-generativeai
python-multipart
```

.env

```
GEMINI_API_KEY=your_gemini_api_key
```

Before running the application, confirm that the .env file is located in the project directory and that the API key is valid. For deployment to a public environment, configure environment variables through the hosting platform rather than exposing the key in source control.

17. Functional Testing

Functional testing verifies that each major feature responds correctly to representative student inputs.

Test 1 - Ask a Question

Input: Which is the largest ocean? Expected result: an AI-generated educational answer.

Test 2 - Explain a Topic

Input: Explain Pythagoras Theorem. Expected result: definition, steps, important points, and a simple example.

Test 3 - Generate a Quiz

Input: Pythagoras Theorem. Expected result: three MCQs, each with four options and a correct answer.

Test 4 - Summarize

Input: a long educational paragraph. Expected result: concise revision-friendly content.

Test 5 - Learning Plan

Input: SQL. Expected result: beginner-to-advanced learning path with timelines and practice activities.

Error testing

The system should also be tested with empty input, invalid task values, API failures, malformed quiz JSON, and missing API keys. Appropriate error handling prevents a single failed request from making the application unusable.

18. Exploring EduGenie and Project Challenges

Asking Questions

Students can enter academic or general educational questions and receive concise responses generated by Gemini.

Explanation of Any Topic

The explanation module converts a topic into a structured explanation using simple language, step-by-step content, important points, and examples.

Summarising Long Paragraphs

The summarizer supports quick revision by reducing lengthy educational passages while attempting to preserve important information.

Generating Quizzes

The quiz generator supports active recall by creating multiple-choice questions from supplied content.

Learning Recommendations

The learning-path module provides a structured sequence from beginner to advanced concepts with practice and assessment ideas.

Challenges and Limitations

- AI-generated responses may occasionally contain factual errors or incomplete information and should be reviewed for important academic decisions.
- Quiz output may not always follow the requested JSON structure, so parsing and validation are necessary.
- API availability, latency, quota, and network connectivity can affect response time.
- API keys must be protected from accidental exposure.
- Prompt quality strongly influences the usefulness of generated content.

19. Conclusion and Future Enhancements

EduGenie demonstrates how generative AI can be integrated into a lightweight educational web application. By combining FastAPI, HTML, CSS, Jinja2, Python modules, and Google Gemini, the project provides a single interface for common student learning activities.

The modular architecture makes the system easy to extend. New modules can be added for additional educational tasks without redesigning the complete application.

Future Enhancements

- User accounts and personalized learning history.
- Progress tracking and quiz scores.
- Subject-wise dashboards.
- Support for additional languages.
- Voice-based question input and audio responses.
- Document or PDF-based question answering.
- Improved quiz validation and scoring.
- Teacher/admin dashboards.
- More advanced prompt templates and response validation.
- Cloud deployment with secure environment-variable management.

Final Summary

EduGenie is a practical demonstration of AI-assisted education. It supports question answering, explanations, quizzes, summaries, and personalized learning recommendations through a simple student-friendly interface. The project highlights generative AI integration, backend API development, frontend design, prompt engineering, modular software architecture, and basic deployment practices.

Project Title: EduGenie – Smart Learning Assistant using Google Gemini

Core Technologies: Python, FastAPI, HTML, CSS, Jinja2, Google Gemini API, Uvicorn, Python-dotenv

Primary Goal: Make learning simpler, more interactive, and accessible through generative AI.